

Before

1. SQL Injection (Search)

[Home](#) Logged in as **Bob** (user) [New Post](#) [Export](#) [Admin Logs](#) [Logout](#)

Microblog

#1 · owner_id=2 · 2026-03-03T00:17:45.368640Z

[HelloWolrd](#)

Hello 20260302

[Edit](#)

#2 · owner_id=2 · 2026-03-03T00:17:57.826624Z · **PRIVATE**

[Hello Private](#)

Private content

[Edit](#)

- **Where it is:** app.py inside the index() function.
- **What the impact is:** The application concatenates the user's search query (q) directly into the SQL string. An attacker can inject SQL syntax (e.g., ' OR 1=1 --) to alter the query logic, allowing them to bypass filters, view all records in the database, or potentially extract data from other tables.
- **Invariant Violated:** Untrusted input must not alter SQL structure.
- **Exploit Idea:** Enter ' OR 1=1 -- into the search bar to bypass the title filter and return all database records.
- **Fix Summary:** Replaced string concatenation in app.py with SQLite parameterized queries (?).
- **Test Added:** test_sqli_login_bypass_fails - Asserts that injecting SQL syntax into a form results in a 200 OK error page rather than a successful bypass.

2. IDOR / Broken Access Control - Read (Private Posts)

[Home](#) Logged in as **Bob** (user) [New Post](#) [Export](#) [Admin Logs](#) [Logout](#)

Microblog

#2 · owner_id=2 · 2026-03-03T00:17:57.826624Z · **PRIVATE**

[Hello Private](#)

Private content

[Edit](#)

#1 · owner_id=2 · 2026-03-03T00:17:45.368640Z

[HelloWolrd](#)

Hello 20260302

[Edit](#)

- **Where it is:** app.py inside the index() (main feed) and view_post() functions.
- **What the impact is:** The SQL query fetches posts without checking if the requester is authorized to see them. A logged-in user (Bob) can view "Private" posts belonging to other users (Alice) either on the main feed or by directly visiting the URL GET /post/.
- **Invariant Violated:** Private data must not be disclosed to unauthorized users.
- **Exploit Idea:** A logged-in user simply navigates to the main feed, or directly visits the URL /post/ of a private post belonging to another user, and the server serves the content without verifying authorization.
- **Fix Summary:** Added an authorization check to both the index() feed and view_post() route to verify current_user.id == post.owner_id before rendering private content.
- **Test Added:** Verified fix by logging in as Bob and confirming Alice's private posts were successfully hidden from the feed.

3. IDOR / Broken Access Control - Write (Editing / Deleting)

Testing, Security

[Home](#) Logged in as **Bob** (user) [New Post](#) [Export](#) [Admin Logs](#) [Logout](#)

Microblog

Search posts by title (vulnerable)

#2 · owner_id=2 · 2026-03-03T00:17:57.826624Z · **PRIVATE**

[Hello Private](#)

Private content

[Edit](#)

#1 · owner_id=2 · 2026-03-03T00:17:45.368640Z

[HelloWoird_Bob](#)

Hello 20260302

[Edit](#)

- **Where it is:** app.py inside the edit_post() and delete_post() functions.
- **What the impact is:** The routes check if a user is logged in, but fail to check if the current user is the **owner** of the post. An attacker can change the post_id in the URL or the form request to modify or delete content belonging to any other user.
- **Invariant Violated:** Users can modify only resources they own (unless admin).
- **Exploit Idea:** An attacker clicks "Edit" on their own post, but intercepts the request or modifies the post_id in the URL to point to a post owned by a different user.
- **Fix Summary:** Implemented an ownership check (if row[1] != u.id and u.role != 'admin': abort(403)) in the edit_post and delete_post routes directly after fetching the post from the database.
- **Test Added:** test_idor_edit_blocked - Logs in as Bob, attempts to send a POST request to edit a post owned by Alice, and asserts that the server correctly blocks it by returning a 403 Forbidden status code.

4. Sensitive Information Disclosure (Credential Logging)

[Home](#) Logged in as **Bob** (user) [New Post](#) [Export](#) [Admin Logs](#) [Logout](#)

Admin Logs

This page is intended for admins (but audit it!).

ID	Event	Detail	Time
8	export	user=Bob path=/workspace/secure_audit_lab/data/Bob.json	2026-03-03T00:23:20.543644Z
7	login_attempt	user=Bob password=bob12345	2026-03-03T00:18:40.902793Z
6	register	new_user=Bob	2026-03-03T00:18:36.031045Z
5	create_post	user=Alice title=Hello Private	2026-03-03T00:17:57.840909Z
4	create_post	user=Alice title=HelloWoIrd	2026-03-03T00:17:45.371395Z
3	login_attempt	user=Alice password=alice12345	2026-03-03T00:17:14.675334Z
2	login_attempt	user= password=alice12345	2026-03-03T00:16:56.787182Z
1	register	new_user=Alice	2026-03-03T00:16:36.868716Z

- **Where it is:** app.py inside the login() and register() functions.
- **What the impact is:** The application explicitly logs the plaintext password into the audit_logs database table (and console) via log_event. If the logs are viewed, every user account is instantly compromised.
- **Invariant Violated:** Secrets must never be logged.
- **Exploit Idea:** View the plaintext server logs or the database audit_logs table to steal the plaintext passwords of all registered users.
- **Fix Summary:** Removed the password={password} string from the log_event() calls in the login and register routes. Additionally, replaced plaintext password storage with werkzeug.security password hashing.
- **Test Added:** Verified fix by checking the Admin Logs page and confirming new login attempts do

- **Test Added:** Verified by checking the Admin Logs page and confirming new login attempts do not display passwords.

5. Broken Access Control (Admin Route)

[Home](#) Logged in as **Bob** (user) [New Post](#) [Export](#) [Admin Logs](#) [Logout](#)

Admin Logs

This page is intended for admins (but audit it!).

ID	Event	Detail	Time
8	export	user=Bob path=/workspace/secure_audit_lab/data/Bob.json	2026-03-03T00:23:20.543644Z
7	login_attempt	user=Bob password=bob12345	2026-03-03T00:18:40.902793Z
6	register	new_user=Bob	2026-03-03T00:18:36.031045Z
5	create_post	user=Alice title=Hello Private	2026-03-03T00:17:57.840909Z
4	create_post	user=Alice title=HelloWoIrd	2026-03-03T00:17:45.371395Z
3	login_attempt	user=Alice password=alice12345	2026-03-03T00:17:14.675334Z
2	login_attempt	user= password=alice12345	2026-03-03T00:16:56.787182Z
1	register	new_user=Alice	2026-03-03T00:16:36.868716Z

- **Where it is:** app.py inside the admin_logs() function.
- **What the impact is:** The route handles the logic to display sensitive logs but lacks a check for current_user.role == 'admin'. Any standard user (like Bob) can access /admin/logs, leading to Privilege Escalation and the leakage of sensitive system data.
- **Invariant Violated:** Private data must not be disclosed to unauthorized users.
- **Exploit Idea:** A low-privileged user manually types the /admin/logs endpoint into their browser's URL bar to access sensitive system logs.
- **Fix Summary:** Added a role-verification check (if not u or u.role != 'admin': abort(403)) at the very beginning of the admin_logs() function to strictly enforce the admin boundary.
- **Test Added:** Verified by attempting to access the route as Bob and receiving a 403 Forbidden response.

6. Weak Session Handling (Forgeable Cookie)

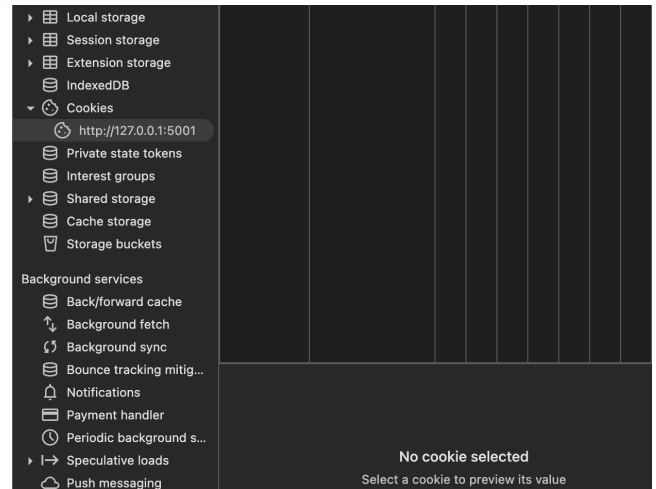
[Home](#) Logged in as **Alice** (user) [New Post](#) [Export](#) [Admin Logs](#) [Logout](#)

Export

Export written to: /workspace/secure_audit_lab/data/Alice.json

Application									
Filter									
Name	Value	D...	P...	E...	Si...	H...	S...	S...	
USER_ID	2	1...	/	S...	8				
Storage									

```
{
  "user": {
    "id": 2,
    "username": "Alice",
    "role": "user"
  },
  "posts": [
    {
      "id": 2,
      "title": "Hello Private",
      "content": "Private content",
      "is_private": true,
      "created_at": "2026-03-03T00:17:57.826624Z"
    },
    {
      "id": 1,
      "title": "HelloWorld_Bob",
      "content": "Hello 20260302",
      "is_private": false,
      "created_at": "2026-03-03T00:17:45.368640Z"
    }
  ]
}
```



- **Where it is:** app.py (login route) and auth.py (current_user function).
- **What the impact is:** The application uses a raw, unencrypted cookie (USER_ID=2) to identify users. Because there is no cryptographic signature, an attacker can modify this integer in their browser to impersonate any user (Account Takeover) without knowing their password.
- **Invariant Violated:** Identity tokens must be unforgeable and integrity-protected.
- **Exploit Idea:** Open the browser's Developer Tools, navigate to the Application/Storage tab, and change the plaintext USER_ID cookie to another user's ID number to instantly take over their account.
- **Fix Summary:** Removed the insecure manual cookie implementation (set_cookie('USER_ID')) and replaced it with Flask's built-in session dictionary, which cryptographically signs the cookie using the app's SECRET_KEY to prevent tampering.
- **Test Added:** Enforced across all state-changing tests implicitly. Furthermore, test_csrf_token_required_for_post was added to ensure state-changing actions require a CSRF token tied to this secure session, asserting a 403 status code if missing.

After

SQL Injection (Search)

[Home](#) Logged in as **Bob** (user) [New Post](#) [Export](#) [Admin Logs](#) [Logout](#)

Microblog

' OR 1=1 --

Search

No posts yet.

Vulnerability Fixed: SQL Injection in Search.

Broken Access Control (Admin Route)

Forbidden

You don't have the permission to access the requested resource. It is either read-protected or not readable by the server.

Vulnerability Fixed: Added a role check to ensure `current_user.role == 'admin'` before loading the page.

IDOR Access Control

[Home](#) Logged in as **Bob** (user) [New Post](#) [Export](#) [Admin Logs](#) [Logout](#)

Microblog

Search posts by title (vulnerable)

Search

#2 · owner_id=2 · 2026-03-04T21:41:08.299646Z · **PRIVATE**

[HelloWorld_Private](#)

Hello World!

[Edit](#) [Delete](#)

#1 · owner_id=2 · 2026-03-04T21:40:52.304428Z

[HelloWorld](#)

Hello 20260304

[Edit](#) [Delete](#)

Application									
Filter									
Name	Value	D...	P...	E...	Si...	H...	S...	S...	
session	eyJjc3JmX3Rva2...	1...	/	S...	124	✓			
USER_ID	1	1...	/	S...	8				

Storage

Local storage

Session storage

Extension storage

IndexedDB

Cookies

http://127.0.0.1:5001

Private state tokens

Interest groups

Shared storage

Cache storage

Storage buckets

Background services

Back/forward cache

Background fetch

Background sync

